LLM Engineering Assignment - Theory

Name: N RITHIKA MARY
EMP ID: BSC027

1. What is an LLM?

An LLM is a type of AI model that learns from a large amount of text. It understands language patterns and generates human-like responses.

2. Why is it called large?

It is trained on huge datasets and has billions of parameters. This helps it understand and generate better text.

3. Evolution of language models

Language models started with rule-based systems, then statistical models, RNNs/LSTMs, and finally Transformers, which are faster and understand context better.

4. Limitations of RNNs/LSTMs

They process words one by one, making them slow, and they find it difficult to remember long sentences.

5. Self-attention

Self-attention helps each word focus on other important words in the sentence. This improves understanding of context.

6. Three limitations

Hallucination – check facts. Knowledge cutoff – use updated data. Context window limit – split long text into smaller parts.

7. Open vs Closed LLMs

Open-weight: Llama, Mistral (can be customized). Closed: ChatGPT, Claude (easy to use but less control).

8. Why tokenization?

Computers understand numbers, not text. Tokenization converts text into tokens and token IDs.

9. Types of tokenization

Character: flexible but long.

Word: simple but misses new words.

Subword: handles both common and new words well.

10. BPE

BPE starts with characters and keeps merging common pairs to create useful subwords.

11. WordPiece vs SentencePiece

WordPiece splits words into subwords. SentencePiece works directly on raw text and is useful for multilingual models.

12. Special tokens

Special tokens like [CLS], [SEP], [PAD] and [MASK] help the model identify sentence start, separation, padding and masked words.

13. Attention mask

It tells the model which tokens are real and which are only padding.

14. Why embeddings?

Token IDs are just numbers. Embeddings convert them into meaningful vectors.

15. One-hot vs dense

One-hot vectors are large and sparse. Dense embeddings are smaller and capture word meaning.

16. Embedding layer

It works like a lookup table. Before training, embeddings are random and improve during training.

17. Input vs output embeddings

Input embeddings represent input words. Output embeddings help predict the next word. Weight tying shares the same weights.

18. Positional embeddings

They tell the model the order of words, so sentence meaning is preserved.

19. Contextual embeddings

The meaning of 'bank' changes based on the sentence, so its embedding also changes.

20. Cosine similarity

1 means very similar, 0 means unrelated, and -1 means opposite.

21. Query, Key, Value

Query asks what to look for, Key matches it, and Value provides the information.

22. Scaled dot-product attention

Multiply Q and K, divide by $\sqrt{dk}$, apply softmax, then multiply by V to get the output.

23. Why scale?

Scaling prevents very large values and makes softmax work better.

24. Multi-head attention

Different heads learn different relationships in the same sentence.

25. Multiple heads

The embedding is split into smaller parts, processed separately, then combined again.

26. Residual & LayerNorm

Residual connections keep important information. LayerNorm makes training stable.

27. FFN & Dropout

FFN learns complex patterns. Dropout helps prevent overfitting.

28. Encoder vs Decoder

The encoder understands input, while the decoder generates text. GPT uses only the decoder.

29. Masked attention

It hides future words so GPT predicts one word at a time.

30. Autoregressive generation

GPT predicts the next word, adds it to the sentence, and repeats until the response is complete.

31. KV Cache

KV Cache stores previous attention values so the model generates text faster without recalculating everything.